Big Data Storage and Retrieval Report

1 Abstract

This report presents the work done for the Big Data Storage and Retrieval assignment. The dataset was cleaned, modeled, and loaded into PostgreSQL, MongoDB, and Neo4j. After that, three analysis tasks and the benchmark tasks were completed for all three systems. The results show clear differences between the models. PostgreSQL worked well for structured relational analysis, MongoDB was strong for the text search task, and the graph model gave the fastest benchmark results overall in this project.

2 Introduction

The goal of this assignment was to compare different database models for an e-commerce dataset that contains customer events, campaigns, campaign messages, first purchase dates, and friendship links between users. The work followed three main parts from the assignment:

- data modeling and storage,
- data analysis,
- benchmarking.

The cleaned dataset used in this work contains the following files:

- `campaigns.csv`: 1,907 rows,
- `client_first_purchase_date.csv`: 175,780 rows,
- `events.csv`: 1,277,717 rows,
- `friends.csv`: 1,974,234 rows,
- `messages.csv`: 2,994,126 rows.

The experiments were run on a laptop with Microsoft Windows 11 Pro, an Intel Core i5-9300H CPU at 2.40 GHz, 4 cores and 8 logical processors, and about 8 GB RAM. PostgreSQL, MongoDB, and Neo4j were used locally during the experiments. The exact software version numbers were not saved in the repository during the experiments, so they are not reported here.

3 Data Modeling

3.1 Data Cleaning

The cleaning step was implemented in `scripts/clean_data.py`. The script reads the raw CSV files and writes cleaned files to `data/processed`. The main cleaning actions were:

- trimming spaces,

- replacing common missing tokens such as `na`, `null`, and empty strings with empty values,
- normalizing Boolean values into a consistent form,
- normalizing date and datetime fields,
- normalizing numeric values,
- skipping malformed rows with extra unnamed columns,
- skipping fully empty rows.

This step was important because the same cleaned files were used by all database loaders, which helped make the comparison fair.

3.2 PostgreSQL Data Model

The PostgreSQL model was implemented in `scripts/load_data_psql.sql`. A relational design was chosen because the dataset has clear entities and many relationships that fit well into tables with keys and constraints.

The main tables are:

- `users`,
- `clients`,
- `products`,
- `events`,
- `campaigns`,
- `messages`,
- `friends`.

The loader first creates staging tables, imports the cleaned CSV files, and then fills the final tables. Primary keys, foreign keys, and a unique key on `(campaign_id, campaign_type)` were used to keep the data consistent. This model is normalized and easy to query with joins, which is useful for the analysis tasks.

3.3 MongoDB Data Model

The MongoDB model was implemented in `scripts/load_data_mongodb.js`. A document model was chosen to reduce the need for many joins and to keep related values together inside documents when that made sense.

The main MongoDB collections are:

- `users`,
- `clients`,
- `products`,
- `events`,
- `campaigns`,
- `messages`,

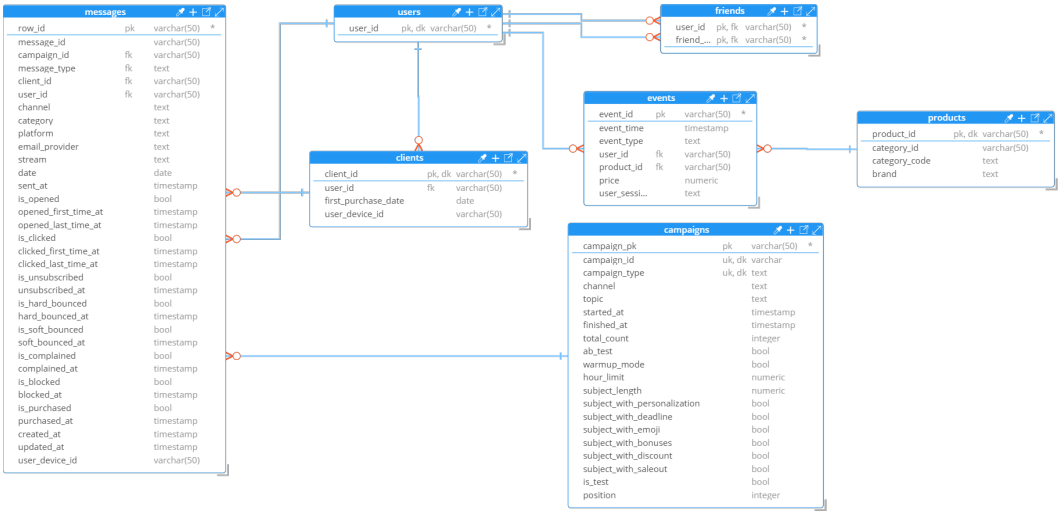

Figure 1: PostgreSQL data model

- `friendships`.

Some values were grouped into sub-documents. For example, campaign information inside messages is stored as `campaign_ref`, and message actions are stored in an `engagement` object. Indexes were added for fields such as user IDs, product IDs, campaign keys, and dates. This design helped MongoDB perform well on some tasks, especially the text search task.

3.4 Neo4j Data Model

The graph model was implemented in `scripts/load_data_graph.cypherl`. The design uses graph nodes such as `User`, `Client`, `Campaign`, `Message`, `Product`, and `Category`, with relationships between them.

This model is a good fit for friendship links and recommendation tasks because social connections are first-class relations in a graph database. In the final version of the work, the graph model was also used for the analysis tasks and the benchmark tasks, so the experimental results in this report compare all three models.

4 Experimental Results

4.1 Analysis Tasks

The analysis was implemented with Python clients in:

- `scripts/analyze_psql.py`,
- `scripts/analyze_mongodb.py`,
- `scripts/analyze_graph.py`.

The full query files are:

- `scripts/q1.sql` and `scripts/q1.js`,
- `scripts/q2.sql` and `scripts/q2.js`,

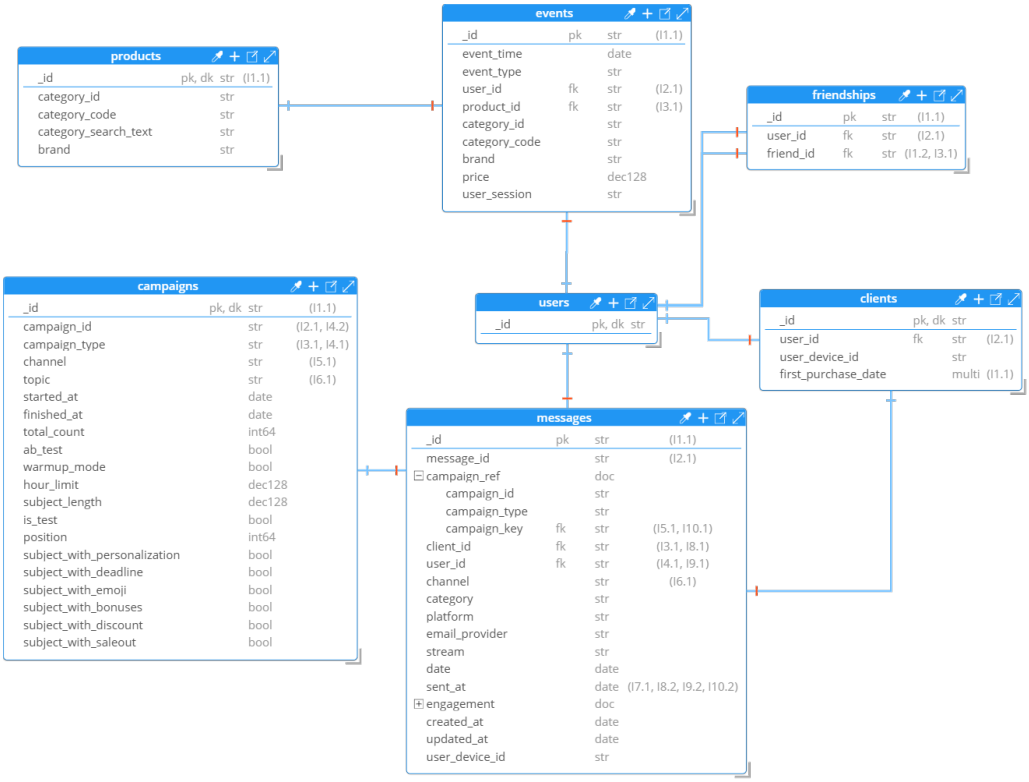

Figure 2: MongoDB data model

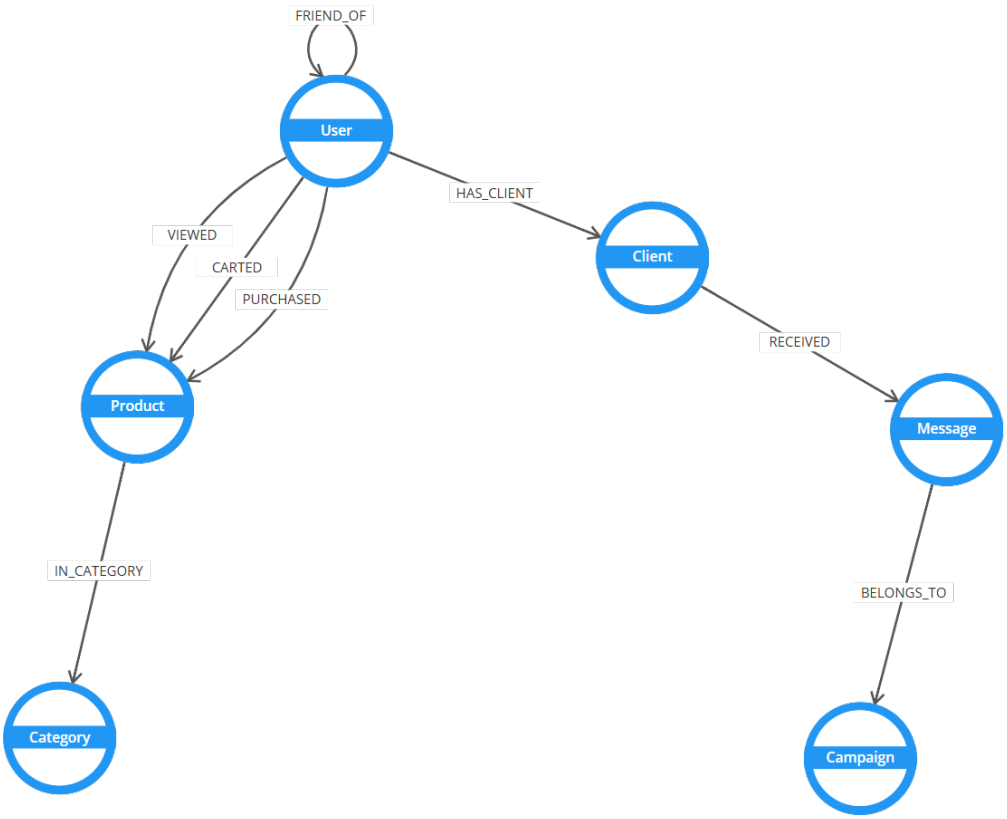

Figure 3: Neo4j data model

- `scripts/q3.sql` and `scripts/q3.js`,
- graph query versions in `scripts/q1.cypherl`, `scripts/q2.cypherl`, and `scripts/q3.cypherl`.

4.1.1 Q1: Did campaigns attract purchases?

Q1 studies whether campaigns led users to purchases. The query first groups messages by campaign and user, then checks which users purchased. After that, it uses the friendship data to find how many purchasers also had at least one friend who purchased in the same campaign. This gives a social support signal.

All three systems showed that transactional email campaigns were among the strongest campaigns in the dataset. In PostgreSQL and MongoDB, campaign `33` with topic `order pickup still pending` had the highest conversion rate at 2.36%. In the graph result, campaign `29` with topic `order cancelled` was ranked first with a conversion rate of 2.16%, and campaign `33` was still near the top with 2.11%.

This suggests that transactional campaigns were more effective than bulk campaigns in this dataset. It also suggests that social links may help in some cases, but the effect is not large in most of the top campaigns.

Table 1: Q1 execution summary

Database	Time (sec)	Rows returned
PostgreSQL	18.24	20
MongoDB	237.08	20
Graph	2.88	20

4.1.2 Q2: Personalized product recommendation

Q2 builds product recommendations for the home page. The idea is simple: take the top 100 active users, look at the products their friends interacted with, give a higher weight to purchases and cart actions, remove products that the target user already touched, and keep the top 5 products for each target user.

All three systems returned 140 recommendation rows. One clear example is user `550133755`, for whom product `1005124` had the highest score of 59 based on 37 friend interactions. Many top recommendations were smartphones and electronics products, which shows that these categories were very active in the dataset. The graph model gave the same task result size, but with much lower execution time than the other two systems.

The result is useful for the business because it gives a practical way to show social-based recommendations on the home page.

Table 2: Q2 execution summary

Database	Time (sec)	Rows returned
PostgreSQL	4.07	140
MongoDB	3.41	140
Graph	0.73	140

4.1.3 Q3: Full-text search on top product categories

Q3 takes the top recommended products from Q2, extracts common keywords from their `category_code`, and then searches products using these keywords. In MongoDB, the keywords

used were `appliances`, `electronics`, and `smartphone`. In the graph result, the keywords included `apparel`, `electronics`, and `smartphone`. MongoDB returned 60 rows for this task, and the graph model returned 58 rows. Both were very fast.

In PostgreSQL, the stored analysis result for Q3 returned 0 rows. So, for this dataset and implementation, MongoDB and the graph model gave useful output, while the PostgreSQL version did not return matching products in the saved run. This is an important difference between the three systems in this project.

Table 3: Q3 execution summary

Database	Time (sec)	Rows returned
PostgreSQL	3.75	0
MongoDB	0.50	60
Graph	0.11	58

4.2 Benchmarking

The benchmark data was saved in:

- `benchmark_results/postgres_results.csv`,
- `benchmark_results/mongo_results.csv`,
- `benchmark_results/graph_results.csv`.

According to the assignment, each query should be run 5 times. This was completed for PostgreSQL, MongoDB, and the graph model, so the benchmark comparison in this version of the report is based on five runs for all three systems.

4.2.1 Raw benchmark results

Table 4: Raw benchmark times for PostgreSQL

Query	Run 1	Run 2	Run 3	Run 4	Run 5
Q1	36686.850	47662.177	23926.330	16823.853	15065.040
Q2	5485.536	3507.523	3526.006	3806.173	3713.116
Q3	4206.594	4181.127	4478.622	4828.611	4522.320

Table 5: Raw benchmark times for MongoDB

Query	Run 1	Run 2	Run 3	Run 4	Run 5
Q1	280689.304	317953.114	278728.113	288223.101	351379.654
Q2	7457.246	2515.047	2397.721	2569.360	3190.586
Q3	227.228	66.798	74.710	65.263	61.631

4.2.2 Average and standard deviation

The benchmark summary shows that MongoDB Q1 stayed much slower than the other two systems in all runs. The graph model had the best average time in all three benchmark queries. MongoDB was also very strong in Q3, while PostgreSQL was clearly slower in Q1 and Q3 than the graph model.

Table 6: Raw benchmark times for the graph model

Query	Run 1	Run 2	Run 3	Run 4	Run 5
Q1	3183.217	3085.288	2750.878	2703.134	3550.405
Q2	744.436	676.222	665.882	623.049	643.021
Q3	79.866	63.261	59.569	87.442	54.875

Table 7: Benchmark summary statistics

Database	Query	Runs	Average (ms)	Std. dev. (ms)
PostgreSQL	Q1	5	28032.850	13882.286
PostgreSQL	Q2	5	4007.671	835.703
PostgreSQL	Q3	5	4443.455	264.915
MongoDB	Q1	5	303394.657	31099.261
MongoDB	Q2	5	3625.992	2163.759
MongoDB	Q3	5	99.126	71.770
Graph	Q1	5	3054.584	346.079
Graph	Q2	5	670.522	46.172
Graph	Q3	5	69.003	13.960

4.2.3 Execution time chart

4.2.4 Benchmark screenshots

5 Discussion

The results show clear strengths and weaknesses for the three tested systems.

PostgreSQL was strong for structured relational analysis. Its schema was normalized, clear, and stable. Q1 and Q2 worked correctly, and the benchmark results were based on five runs. The main weakness in this work was Q3, where the saved run returned no rows.

MongoDB was flexible and performed very well for the text search task. Q3 was especially strong in MongoDB, both in result quality and speed. Q2 was also a bit faster than PostgreSQL on average, but it had larger variation between runs. However, MongoDB had a major weakness in Q1. That query needed a large amount of processing over messages and friendship links, and in all five runs it was much slower than PostgreSQL and the graph model.

The graph model performed very well in this project. It was the fastest system in all three benchmark queries, and it also returned useful results for Q1, Q2, and Q3. This matches the idea behind graph databases: they are well suited for linked data, recommendation tasks, and relation-heavy queries.

The campaign analysis also gives a business insight. Transactional email campaigns had the best conversion rates in the top results. Bulk campaigns appeared in the ranking, but they were not the strongest ones. The social support signal was present, but usually small, so it should be treated as an extra feature and not the only rule for future targeting.

Average execution times (ms)
PostgreSQL: Q1 = 28032.850, Q2 = 4007.671, Q3 = 4443.455
MongoDB: Q1 = 303394.657, Q2 = 3625.992, Q3 = 99.126
Graph: Q1 = 3054.584, Q2 = 670.522, Q3 = 69.003

Figure 4: Text summary of average query execution times

```
mostafa@Mostafa:/mnt/e/git_repos/big-data-storage-and-retrieval$ scripts/benchmark_psql.py 5
Benchmarking PostgreSQL: customer_campaign_analytics @ localhost:5432 as postgres
Runs per query: 5
[q1] run 1/5: 36686.850 ms
[q1] run 2/5: 47662.177 ms
[q1] run 3/5: 23926.330 ms
[q1] run 4/5: 16823.853 ms
[q1] run 5/5: 15065.040 ms
[q1] average: 28032.850 ms
[q2] run 1/5: 5485.536 ms
[q2] run 2/5: 3507.523 ms
[q2] run 3/5: 3526.006 ms
[q2] run 4/5: 3806.173 ms
[q2] run 5/5: 3713.116 ms
[q2] average: 4007.671 ms
[q3] run 2/5: 4181.127 ms
[q3] run 3/5: 4478.622 ms
[q3] run 4/5: 4828.611 ms
[q3] run 5/5: 4522.320 ms
[q3] average: 4443.455 ms
Saved benchmark results: /mnt/e/git_repos/big-data-storage-and-retrieval/benchmark_results/postgres_results.csv
```

Figure 5: PostgreSQL benchmark screenshot

```
mostafa@Mostafa:/mnt/e/git_repos/big-data-storage-and-retrieval$ scripts/benchmark_mongo.py 5
Benchmarking MongoDB: customer_campaign_analytics @ mongodb://localhost:27017
Runs per query: 5
[23:34:43] q1: ensuring helper indexes.
[23:34:43] q1: building campaign-user purchase table.
[23:39:42] q1: campaign-user table ready (2298764 rows).
[23:39:42] q1: loading friendship graph into memory.
[23:39:52] q1: friendship graph loaded (656455 users with at least one friend).
[23:39:52] q1: computing social purchaser counts.
[q1] run 1/5: 280689.304 ms
[23:39:53] q1: ensuring helper indexes.
[23:39:53] q1: building campaign-user purchase table.
[23:45:07] q1: campaign-user table ready (2298764 rows).
[23:45:07] q1: loading friendship graph into memory.
[23:45:47] q1: friendship graph loaded (656455 users with at least one friend).
[23:45:47] q1: computing social purchaser counts.
[q1] run 2/5: 317953.114 ms
[23:45:48] q1: ensuring helper indexes.
[23:45:48] q1: building campaign-user purchase table.
[23:50:37] q1: campaign-user table ready (2298764 rows).
[23:50:37] q1: loading friendship graph into memory.
[23:50:55] q1: friendship graph loaded (656455 users with at least one friend).
[23:50:55] q1: computing social purchaser counts.
[q1] run 3/5: 278728.113 ms
[23:50:57] q1: ensuring helper indexes.
[23:50:57] q1: building campaign-user purchase table.
[23:55:54] q1: campaign-user table ready (2298764 rows).
[23:55:54] q1: loading friendship graph into memory.
[23:56:14] q1: friendship graph loaded (656455 users with at least one friend).
[23:56:14] q1: computing social purchaser counts.
[q1] run 4/5: 288223.101 ms
[23:56:15] q1: ensuring helper indexes.
[23:56:15] q1: building campaign-user purchase table.
[00:02:15] q1: campaign-user table ready (2298764 rows).
[00:02:15] q1: loading friendship graph into memory.
[00:02:45] q1: friendship graph loaded (656455 users with at least one friend).
[00:02:45] q1: computing social purchaser counts.
[q1] run 5/5: 351379.654 ms
[q1] average: 303394.657 ms
[00:02:46] q2: selecting top 100 active users from events.
[00:02:51] q2: found 100 target users, running recommendation aggregation.
[00:02:53] q2: aggregation finished.
```

Figure 6: MongoDB benchmark screenshot

```
mostafa@Mostafa:/mnt/e/git_repos/big-data-storage-and-retrieval$ venv/bin/python scripts/benchmark_graph.py 5
Benchmarking Graph DB: neo4j @ bolt://localhost:7687 as neo4j
Runs per query: 5
[q1] run 1/5: 3183.217 ms
[q1] run 2/5: 3085.288 ms
[q1] run 3/5: 2750.878 ms
[q1] run 4/5: 2703.134 ms
[q1] run 5/5: 3550.405 ms
[q1] average: 3054.584 ms
[q2] run 1/5: 744.436 ms
[q2] run 2/5: 676.222 ms
[q2] run 3/5: 665.882 ms
[q2] run 4/5: 623.049 ms
[q2] run 5/5: 643.021 ms
[q2] average: 670.522 ms
[q3] Keywords from latest q2 result: ['electronics', 'apparel', 'smartphone']
[q3] run 1/5: 79.866 ms
[q3] run 2/5: 63.261 ms
[q3] run 3/5: 59.569 ms
[q3] run 4/5: 87.442 ms
[q3] run 5/5: 54.875 ms
[q3] average: 69.003 ms
Saved benchmark results: /mnt/e/git_repos/big-data-storage-and-retrieval/benchmark_results/graph_results.csv
```

Figure 7: Graph benchmark screenshot

6 Conclusion

In this assignment, three data models were built and tested for the same e-commerce dataset. PostgreSQL, MongoDB, and Neo4j were all used for the analysis tasks and the benchmark tasks. PostgreSQL was a solid relational baseline, MongoDB was strong for text search, and the graph model gave the best benchmark performance overall. Based on the completed experiments, the graph model was the fastest option in this project, while PostgreSQL and MongoDB still showed useful strengths for different kinds of workloads.